

Ultimate Preparation Guide: 2D Continuous Fourier Transform

Part 2: Properties, Code, and Intuition

Prerequisite Utilities: In 2D CFT, your class returns separate ‘real’ and ‘imag’ arrays. It is mathematically much easier to combine them into a single complex array before verifying properties. Add these to your script:

```
import numpy as np
import copy

def calculate_mse(arr1, arr2):
    """Computes the Mean Squared Error between two complex 2D arrays."""
    return np.mean(np.abs(arr1 - arr2)**2)

def get_complex_cft(cft_obj):
    """Computes CFT and returns a single complex 2D array."""
    real, imag = cft_obj.compute_cft()
    return real + 1j * imag

def get_uv_grids(cft_obj):
    """Returns 2D grids for u and v to allow vectorized math on the spectrum."""
    # meshgrid creates 2D arrays from 1D axes.
    # u maps to x (columns), v maps to y (rows).
    U, V = np.meshgrid(cft_obj.u, cft_obj.v)
    return U, V
```

1 Layer 1: Basic Implementations (Adapted Previous Year Questions)

1.1 Question 1: 2D Spatial Shifting (Adapted from PYQ Time-Shift)

Problem: Shift the image spatially by $x_0 = 0.2$ and $y_0 = -0.1$. Compute the CFT of the shifted image and verify the Spatial Shift property.

Theory & Intuition: $I(x - x_0, y - y_0) \leftrightarrow F(u, v)e^{-j2\pi(ux_0 + vy_0)}$. Intuition: Moving an image in space does not change its frequency content (the edges and textures are exactly the same), so the **magnitude spectrum remains identical**. However, the **phase spectrum** shifts linearly to account for the new spatial position.

```
# --- Setup ---
img = ContinuousImage("pikachu.png")
cft_orig = CFT2D(img)
F_orig = get_complex_cft(cft_orig)

x0, y0 = 0.2, -0.1
dx = img.x[1] - img.x[0]
dy = img.y[1] - img.y[0]

# Shift the image array using np.roll (pixel-wise shift)
shift_x_pixels = int(x0 / dx)
shift_y_pixels = int(y0 / dy)
```

```

img_shifted_arr = np.roll(img.image, shift_x_pixels, axis=1)
img_shifted_arr = np.roll(img_shifted_arr, shift_y_pixels, axis=0)

# Hack: Clone the object to pass to CFT2D
img_shifted = copy.deepcopy(img)
img_shifted.image = img_shifted_arr

cft_shifted = CFT2D(img_shifted)
F_shifted = get_complex_cft(cft_shifted)

# --- Verification ---
U, V = get_uv_grids(cft_orig)
# Theoretical prediction
F_theoretical = F_orig * np.exp(-1j * 2 * np.pi * (U * x0 + V * y0))

print(f"Q1 Spatial Shift MSE: {calculate_mse(F_shifted, F_theoretical)}")

# Verify Magnitude is unchanged (as per PYQ requirements)
mse_mag = calculate_mse(np.abs(F_shifted), np.abs(F_orig))
print(f"Q1 Magnitude MSE (Should be ~0): {mse_mag}")

```

1.2 Question 2: Frequency Shifting / Modulation

Problem: Modulate the image by multiplying it by a 2D complex exponential $I_{new}(x, y) = I(x, y)e^{j2\pi(u_0x+v_0y)}$. Verify the property.

Theory & Intuition: $I(x, y)e^{j2\pi(u_0x+v_0y)} \leftrightarrow F(u - u_0, v - v_0)$. Intuition: Multiplying by a wave in the spatial domain shifts the entire frequency spectrum by (u_0, v_0) . This is the foundational math behind radio transmission (shifting a voice signal to a high-frequency radio band).

```

# --- Setup ---
u0, v0 = 5.0, 10.0 # Shift by 5 units in u, 10 units in v
X, Y = np.meshgrid(img.x, img.y)

# Modulate the image
img_mod_arr = img.image * np.exp(1j * 2 * np.pi * (u0 * X + v0 * Y))

# Notice: The image is now COMPLEX. Your CFT2D class assumes a real image.
# If the test asks this, you must compute CFT for real and imag parts separately!
img_real = copy.deepcopy(img); img_real.image = img_mod_arr.real
img_imag = copy.deepcopy(img); img_imag.image = img_mod_arr.imag

cft_real = CFT2D(img_real)
cft_imag = CFT2D(img_imag)

# Linearity: F{Re + j*Im} = F{Re} + j*F{Im}
F_mod = get_complex_cft(cft_real) + 1j * get_complex_cft(cft_imag)

# --- Verification ---
# Theoretical: The spectrum is shifted. We can verify by shifting F_orig.
du = cft_orig.u[1] - cft_orig.u[0]
dv = cft_orig.v[1] - cft_orig.v[0]
shift_u_pixels = int(u0 / du)
shift_v_pixels = int(v0 / dv)

F_theoretical = np.roll(F_orig, shift_u_pixels, axis=1)
F_theoretical = np.roll(F_theoretical, shift_v_pixels, axis=0)

print(f"Q2 Modulation MSE: {calculate_mse(F_mod, F_theoretical)}")

```

2 Layer 2: Harder (Calculus & Scaling)

2.1 Question 3: Partial Derivatives (Adapted from PYQ Derivative)

Problem: Compute the partial derivative of the image with respect to x , $I_x(x, y) = \frac{\partial}{\partial x} I(x, y)$. Verify the derivative property of the CFT.

Theory & Intuition: $\frac{\partial}{\partial x} I(x, y) \leftrightarrow j2\pi u F(u, v)$. Intuition: A derivative highlights areas of rapid change (edges). In the frequency domain, multiplying by u amplifies high frequencies (edges) and zeroes out $u = 0$ (flat backgrounds). This is exactly why derivatives act as high-pass edge detectors!

```
# --- Setup ---
dx = img.x[1] - img.x[0]
# Compute partial derivative w.r.t x (axis=1 is the x-axis/columns)
img_dx_arr = np.gradient(img.image, dx, axis=1)

img_dx = copy.deepcopy(img)
img_dx.image = img_dx_arr

cft_dx = CFT2D(img_dx)
F_dx = get_complex_cft(cft_dx)

# --- Verification ---
U, V = get_uv_grids(cft_orig)
# Theoretical prediction: Multiply original CFT by j * 2 * pi * u
F_theoretical = 1j * 2 * np.pi * U * F_orig

print(f"Q3 Partial Derivative (x) MSE: {calculate_mse(F_dx, F_theoretical)}")
```

2.2 Question 4: Spatial Scaling

Problem: Scale the image spatially by a factor of $a = 2$ (zoom out). Verify the scaling property.

Theory & Intuition: $I(ax, by) \leftrightarrow \frac{1}{|ab|} F(\frac{u}{a}, \frac{v}{b})$. Intuition: If you compress a signal in space (make it smaller/narrower), it changes faster. Therefore, its frequency spectrum must expand (widen) to contain higher frequencies. "Narrow in space = Wide in frequency."

```
# --- Setup ---
a = 2.0
# To scale I(ax, ay), we evaluate the image at scaled coordinates.
# We use np.interp for each row/col, or scipy.ndimage.zoom.
# For simplicity, let's just scale the spatial axes in the object!
img_scaled = copy.deepcopy(img)
img_scaled.x = img.x * a
img_scaled.y = img.y * a

# Because the spatial extent is larger, the frequency resolution changes!
cft_scaled = CFT2D(img_scaled)
F_scaled = get_complex_cft(cft_scaled)

# --- Verification ---
# The theoretical spectrum is scaled in amplitude by 1/a^2
# And the frequency axes themselves are compressed by 1/a.
# Since we let CFT2D recalculate the axes, we just compare amplitudes.
F_theoretical = F_orig * (1 / (a**2))

print(f"Q4 Spatial Scaling MSE: {calculate_mse(F_scaled, F_theoretical)}")
```

3 Layer 3: Very Hard (Gaussian, Reconstructions, & The "Mixed" Fear)

3.1 Question 5: The 2D Gaussian (Adapted from PYQ Gaussian)

Problem: Create a 2D Gaussian image $I(x, y) = e^{-\pi(x^2+y^2)}$. Compute its CFT and verify that the Fourier Transform of a Gaussian is another Gaussian.

Theory & Intuition: $e^{-\pi(x^2+y^2)} \leftrightarrow e^{-\pi(u^2+v^2)}$. Intuition: The Gaussian function is the unique eigenfunction of the Fourier Transform. It is the only shape in the universe that looks exactly the same in both the spatial domain and the frequency domain!

```
# --- Setup ---
X, Y = np.meshgrid(img.x, img.y)
gaussian_arr = np.exp(-np.pi * (X**2 + Y**2))

img_gauss = copy.deepcopy(img)
img_gauss.image = gaussian_arr

cft_gauss = CFT2D(img_gauss)
F_gauss = get_complex_cft(cft_gauss)

# --- Verification ---
U, V = get_uv_grids(cft_gauss)
# Theoretical CFT is also a Gaussian!
F_theoretical = np.exp(-np.pi * (U**2 + V**2))

# Because our spatial domain is truncated to [-1, 1] instead of [-inf, inf],
# there will be a slight truncation error, but MSE should be very low.
print(f"Q5 Gaussian Eigenfunction MSE: {calculate_mse(F_gauss, F_theoretical)}")
```

3.2 Question 6: The "Mixed" Question (FS vs CFT)

Fear Addressed: What if they ask about the relationship between Fourier Series (Task 1) and Fourier Transform (Task 2)?

Theory & Intuition: If you take a single period of a periodic signal (from Task 1) and treat it as a non-periodic signal, you can take its CFT. The CFT of that single period will be a continuous curve. The Fourier Series coefficients (c_n) from Task 1 are exactly equal to the CFT of the single period, **sampled at the harmonic frequencies** ($u = n/T$), scaled by $1/T$. Mathematically: $c_n = \frac{1}{T} F(u) \Big|_{u=\frac{n}{T}}$.

If they ask this, they will provide a 1D CFT class. You would compute the 1D CFT, evaluate it at $u = n/T$, divide by T , and compare it to your c_n array using MSE.

3.3 Question 7: Edge Detection Equivalence (The Ultimate Test)

Problem: In your assignment, you used a circular High-Pass Filter to find edges. Prove that applying the Laplacian operator $\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$ in the spatial domain is equivalent to multiplying the frequency spectrum by $-4\pi^2(u^2 + v^2)$.

Theory & Intuition: Applying the derivative property twice: $\frac{\partial^2}{\partial x^2} \leftrightarrow (j2\pi u)^2 = -4\pi^2 u^2$. Therefore, the Laplacian (which is the standard 2D edge detector in computer vision) is exactly equivalent to a parabolic high-pass filter in the frequency domain!

```
# --- Setup ---
# 1. Spatial Domain Laplacian
dx = img.x[1] - img.x[0]
dy = img.y[1] - img.y[0]
# Second derivatives
I_xx = np.gradient(np.gradient(img.image, dx, axis=1), dx, axis=1)
```

```

I_yy = np.gradient(np.gradient(img.image, dy, axis=0), dy, axis=0)
laplacian_arr = I_xx + I_yy

img_lap = copy.deepcopy(img)
img_lap.image = laplacian_arr
cft_lap = CFT2D(img_lap)
F_lap = get_complex_cft(cft_lap)

# --- Verification ---
# 2. Frequency Domain Parabolic Filter
U, V = get_uv_grids(cft_orig)
F_theoretical = -4 * (np.pi**2) * (U**2 + V**2) * F_orig

print(f"Q7 Laplacian Edge Detection MSE: {calculate_mse(F_lap, F_theoretical)}")

```